

EVENT-DRIVEN & PROCESS-ORIENTED ARCHITECTURES • FS 2026

Peer-based Content Verification Engine

Stream Processing Layer — Assignment 2 — Presentation

Group 4 Evan Martino · Marco Birchler · Roman Babukh

University of St.Gallen · 21.05.2026



Agenda

1

Recap — Peer-based Content Verification & Architecture

2

What's new since the midterm

3

Stream processing layer — overview

4

Processor I — SLA Monitor

5

Processor II — Content Decision Ledger

6

Lecture concepts Coverage

7

Trade-offs

8

Demo & Q&A



Recap — Peer-based Content Verification

Where we left off at the midterm

Decentralised verification of social-media content

Authors submit content → registered peers approve or reject

Verified items receive a cryptographic attestation signature

Reports flow into a moderation track with human review on objection

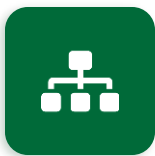
Five bounded contexts · hybrid orchestration (Camunda 8) + choreography (Kafka)

THREE ORCHESTRATED BPMN PROCESSES

1 Register User

2 Verify Content
(peer + internal)

3 Report Content
(with objection)



Recap — Architecture

Services · BPMN orchestration · Kafka event bus

User Identity

`user-service :8001`

Content Verification

`verification-service :8002`

Reporting

`reporting-service :8003`

Attestation

`attestation-service :8004`

Notification

`notification-service`

Camunda 8 / Zeebe

`orchestrator :26500`



Apache Kafka — shared event bus

`user-registered · verification-notification · report-notification · post-deleted · objection-approved`



What's New Since the Midterm

Assignment 2 — Stream Processing with Kafka Streams

1

Dedicated stream-processing layer

Added on top of the existing Kafka event bus — no changes to any BPMN flow

2

Two independent processors

Both consume the same domain topics; Processor I in Python (confluent-kafka), Processor II in Java (Kafka Streams DSL)

3

Assignment 2 coverage

Stateless ops · streams & tables · more than one input stream · interactive queries · windowed operations

4

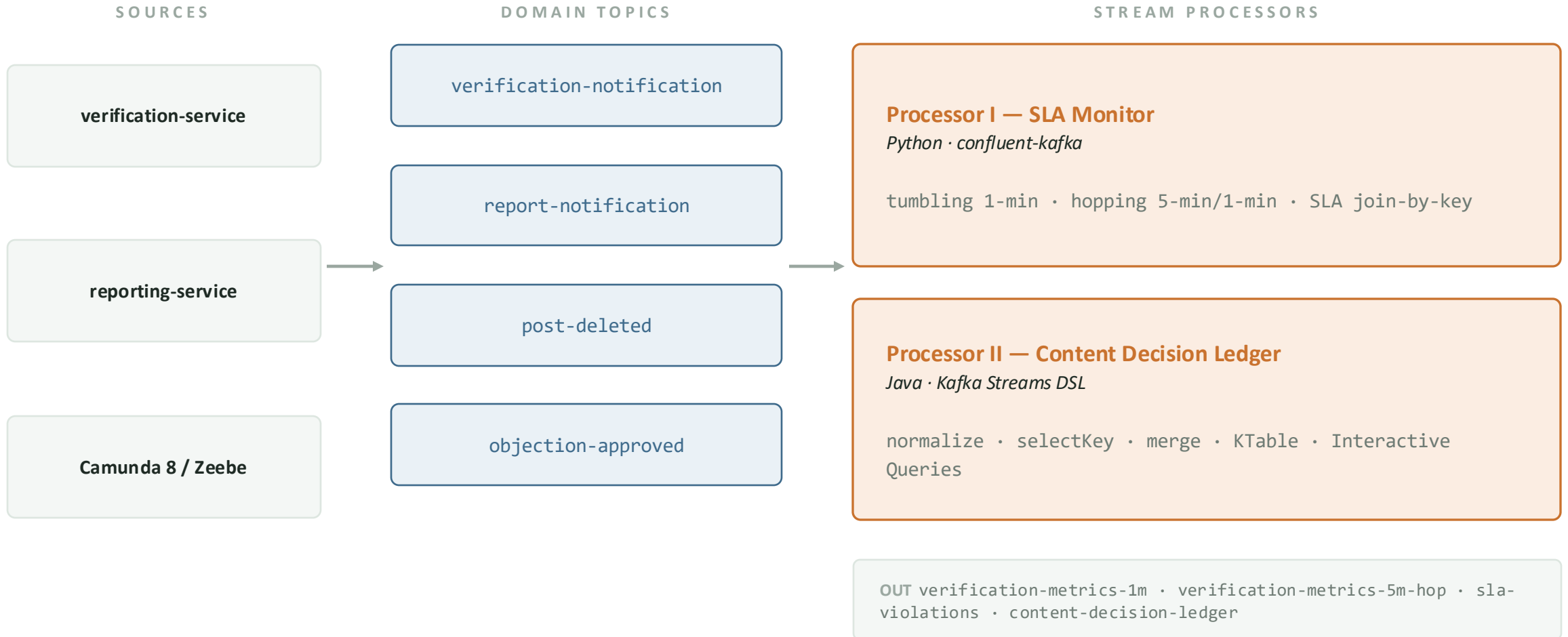
New documentation set

Design proposals, README + Topology diagram



Stream Processing Layer — Overview

Both processors consume the same domain topics — no upstream changes





Processor I — SLA Monitor

Goal · observability for verification + SLA enforcement for moderation

Real-time aggregates on verification outcomes (verified / peer-rejected / internal-rejected / unregistered / timed-out)

Tumbling 1-minute windows for operational dashboards

Hopping 5-minute / 1-minute windows for trend smoothing

Moderation SLA clock: report-accepted → post-deleted v objection-approved within 72 h

Emits a sla-violations event when the clock expires; resolves it if a late outcome arrives

Read-side via Flask: /metrics/verification · /sla/open-cases · /sla/violations

INPUTS (4)

verification-notification
report-notification
post-deleted
objection-approved

OUTPUTS (3)

verification-metrics-1m
verification-metrics-5m-hop
sla-violations

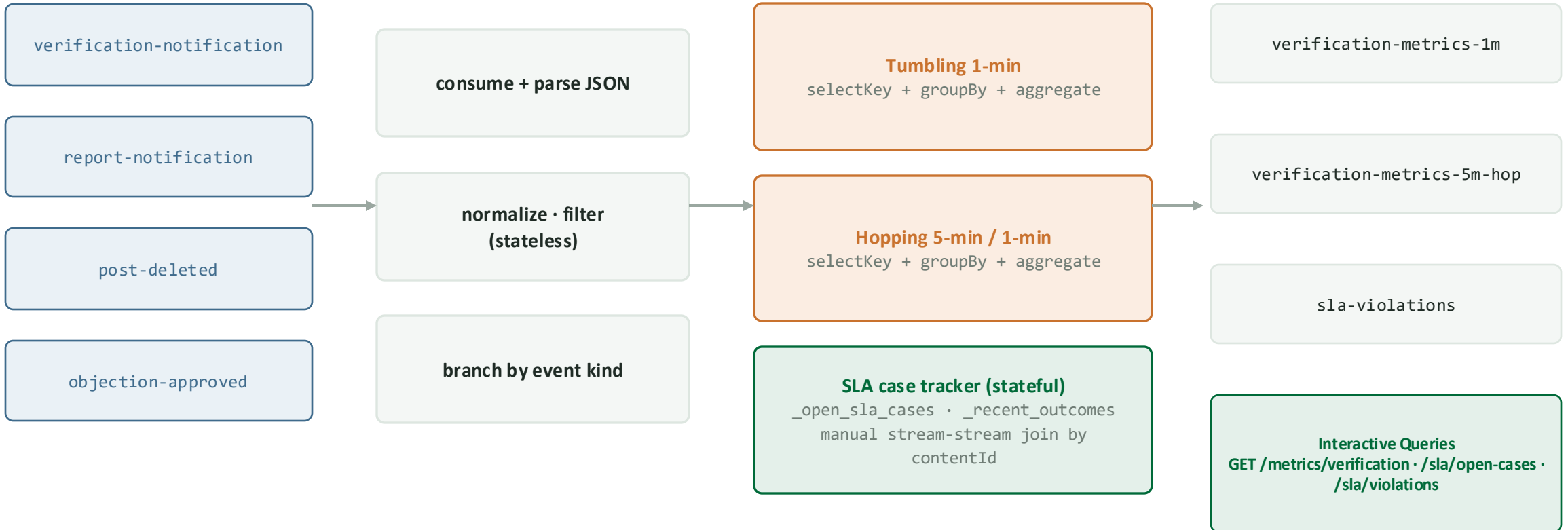
STACK

Python 3.12
confluent-kafka + Flask



Processor I — Topology (SLA Monitor)

Two sub-topologies — verification metrics · SLA case tracking



Note · Implemented in Python with the confluent-kafka API. Maps cleanly to the Kafka Streams DSL: mapValues / filter / branch / windowedBy / aggregate / stream-stream join.



Processor I — Lecture Concepts Realised

Mapping from class material (Weeks 8–10) to the implementation

CONCEPT	WEEK	WHERE IN CODE / TOPOLOGY
Single-Event Processing	Week 8	normalize + filter on every Kafka message before routing
Branch Pattern	Week 8	verification events → metrics path; report/outcome → SLA tracker
Repartitioning by key	Week 8	SLA correlation uses contentId as the implicit key
Stream–Stream Join (manual)	Week 9	report-accepted ⋈ post-deleted / objection-approved by contentId
Local State Store	Week 9	_open_sla_cases · _recent_outcomes · _sla_violations (in-memory)
Interactive Queries	Week 9	Flask endpoints expose the current materialised state directly
Tumbling Window	Week 10	1-minute verification counts + success/failure rates
Hopping Window	Week 10	5-minute window hopping every 1 minute (trend smoothing)
Event-time semantics	Week 10	eventTime parsed from payload, not consumer wall clock
Grace period / late events	Week 10	WINDOW_GRACE_SECONDS + out-of-order outcome buffering
Out-of-order handling	Week 10	Outcome arriving before accept is staged in _recent_outcomes



Processor II — Content Decision Ledger

Goal · per-content audit trail materialised as a KTable

Every event about a content item is folded into a ContentDecisionState

Lifecycle covered: VERIFICATION → REPORT → DELETION / OBJECTION-APPROVED

Materialised KTable backed by RocksDB + Kafka changelog (resilient state)

Output topic content-decision-ledger is log-compacted — replay rebuilds the decision

Two read endpoints serve the table directly via Interactive Queries

Idempotent aggregator: dedup by eventId; trace sorted by eventTime on insert

INPUTS (4 STREAMS)

verification-notification
report-notification
post-deleted
objection-approved

OUTPUTS

content-decision-ledger
(log-compacted topic)
RocksDB: content-state-store

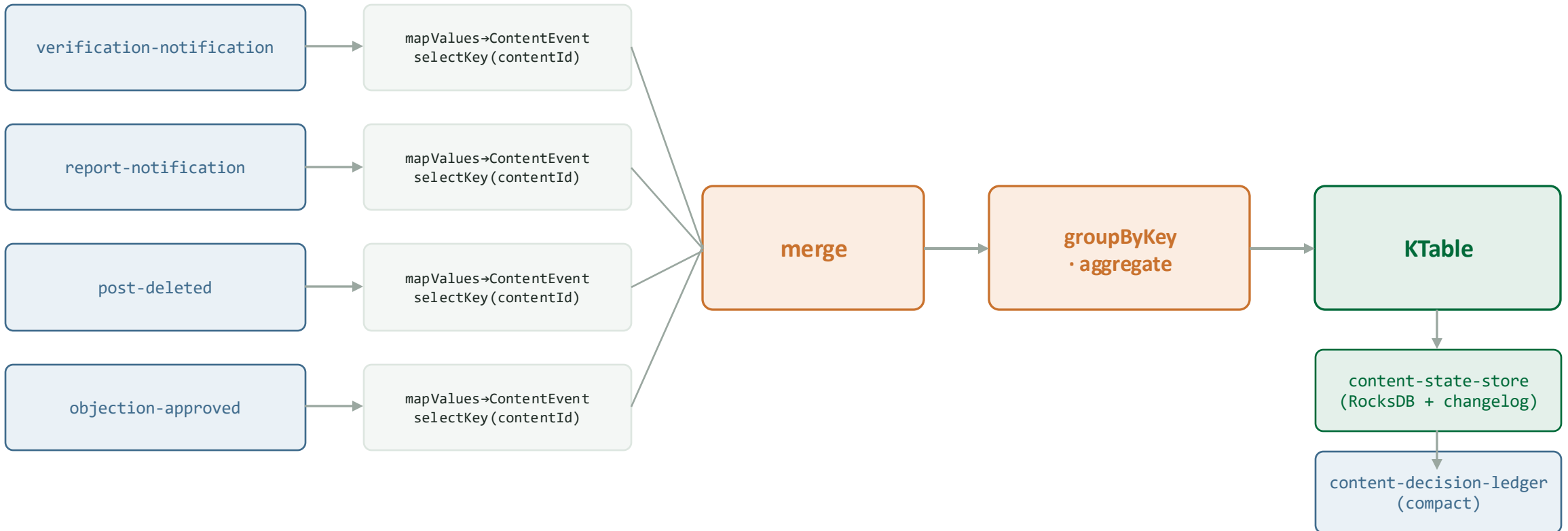
STACK

Java 21 · Spring Boot 3.3
Kafka Streams 3.7 (DSL)



Processor II — Topology

Content Decision Ledger · 4 streams → normalize → re-key → merge → aggregate → KTable → IQ + sink



Interactive Queries (Spring MVC) GET /api/content · /api/content/{id}/state · /api/content/{id}/decision-trace



Processor II — Lecture Concepts Realised

The Kafka Streams DSL maps almost 1-to-1 to the lecture topology vocabulary

CONCEPT	WEEK	WHERE IN CODE / TOPOLOGY
Stateless map/filter	Week 8	EventNormalizer + mapValues per input topic
selectKey → repartitioning	Week 8	selectKey(contentId) co-locates all events per content
Stream merger (4 → 1)	Week 8	verifications.merge(reports).merge(deletions).merge(objections)
Streams AND Tables together	Week 9	KStream sources → KTable aggregate
groupByKey + aggregate	Week 9	Builds ContentDecisionState as the read-side projection
Materialized state (RocksDB)	Week 9	Stores.persistentKeyValueStore("content-state-store")
Interactive Queries	Week 9	LedgerQueryService → KafkaStreams.store(...) → REST
Reprocessing pattern	Week 8	Output topic cleanup.policy=compact → replay rebuilds state
Event-time hygiene	Week 10	PayloadEventTimeExtractor reads eventTime from payload
Idempotency / dedupe	Week 8	Aggregator tracks seenEventIds; duplicates are skipped
Out-of-order safety	Week 10	Decision trace sorted by eventTime on insert



Lecture concepts Coverage

Both processors together cover every suggested item from the task sheet



At least several stateless operations

Week 8

Both processors: normalize / filter / branch / mapValues / selectKey



Usage of both streams and tables

Week 9

Ledger: KStream sources aggregated into a Materialized KTable



Consider data from more than one stream

Week 9

Both: consume 4 input topics — Ledger merges them into one logical stream



Interactive queries

Week 9

SLA: /metrics/verification · /sla/* — Ledger: /api/content/* (local store reads)



Windowed operations

Week 10

SLA Monitor: 1-min tumbling + 5-min hopping windows with grace period



Avro schemes (suggested)

—

Future-proof docs available; runtime uses JSON to match upstream



Trade-offs for the Processing Layer

Four design decisions, made deliberately

1 · Python consumer vs Kafka Streams DSL

- + Python (SLA): lowest setup cost, reuses our stack
- Python (SLA): manual state, windowing, no changelog recovery
- + Kafka Streams DSL (Ledger): declarative topology, RocksDB + changelog
- Kafka Streams DSL (Ledger): heavier toolchain, longer build

2 · JSON over the wire vs Avro + Schema Registry

- lower complexity vs stricter schema enforcement
- JSON for now to reduce complexity
- Avro schemas used as explicit event contracts
- Upgrade path to full Avro already in place

3 · Event time vs processing time

- Use `eventTime` to process when the event happened, not when it arrived
- Enables replayable and out-of-order processing
- Late events need an explicit handling policy

4 · Event driven approach vs a traditional DB

- No (or very little) changes needed in the services
- Replayability - new fields can be aggregated later
- Eventual consistency, not immediate
- + Read scalability and latency
- State is distributed per instance/partition



Demo & Q&A

Live walk-through of the stream-processing layer



Repository github.com/chechmek/EDPO_FS26_Project

Thank you · Questions welcome



Appendix — Contributions

RB

Roman Babukh

Processor I — SLA Monitor

- Python topology · windowed verification metrics
- Stateful SLA case tracker with out-of-order safety
- Flask Interactive-Query endpoints
- Load-generation scripts

MB

Marco Birchler

Processor II — Content Decision Ledger

- Kafka Streams DSL topology (Java 21 / Spring Boot)
- ContentEvent / ContentDecisionState domain model
- RocksDB-backed KTable + log-compacted ledger topic
- TopologyTestDriver test suite + LedgerController IQ API

EM

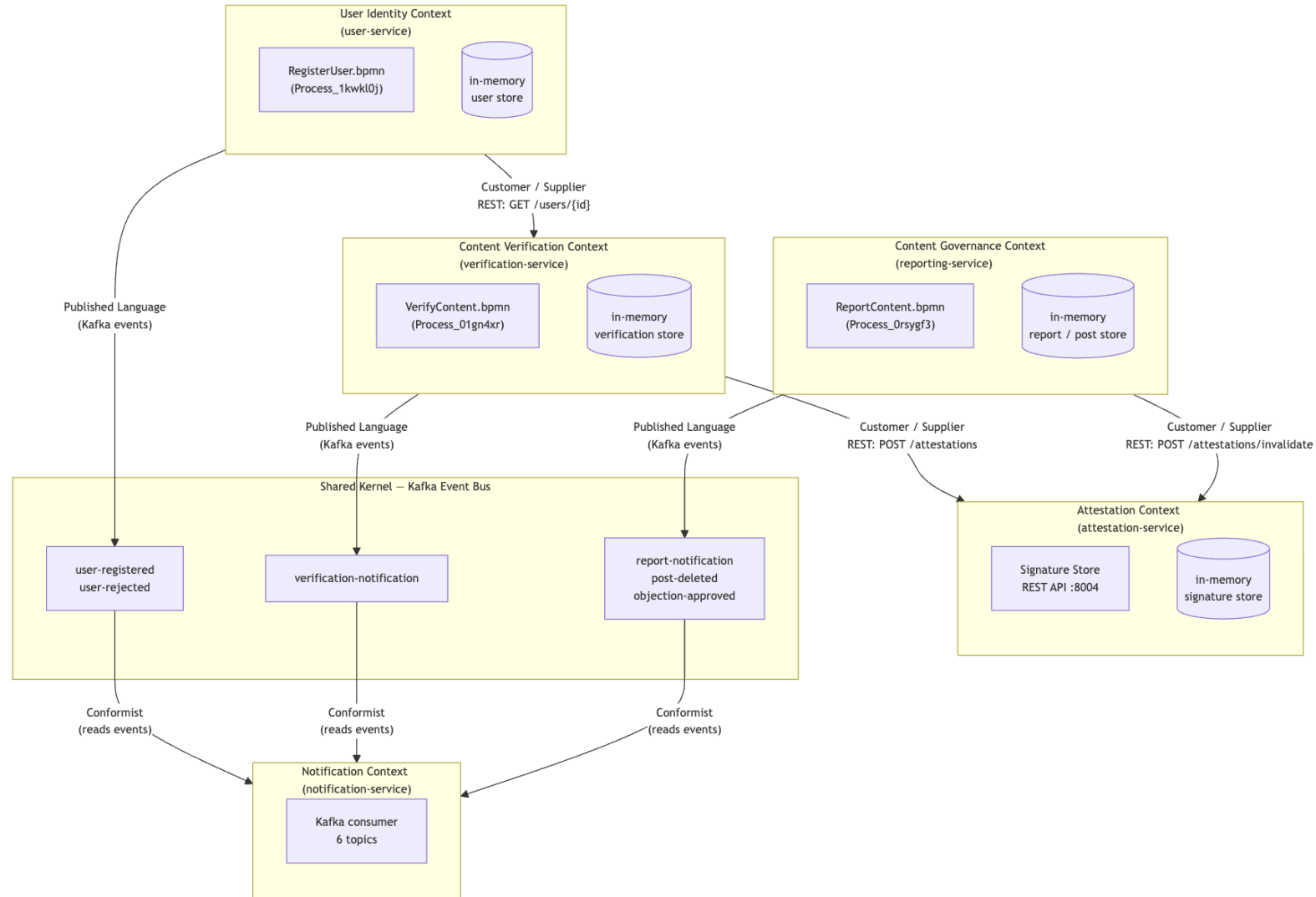
Evan Martino

Support & Documentation

- Design proposals (P1 SLA Monitor · P2 Content Ledger)
- Architecture diagrams, READMEs, TOPOLOGY.md
- Integration & demo scripts (demo.sh, generate_load.py)
- Final presentation

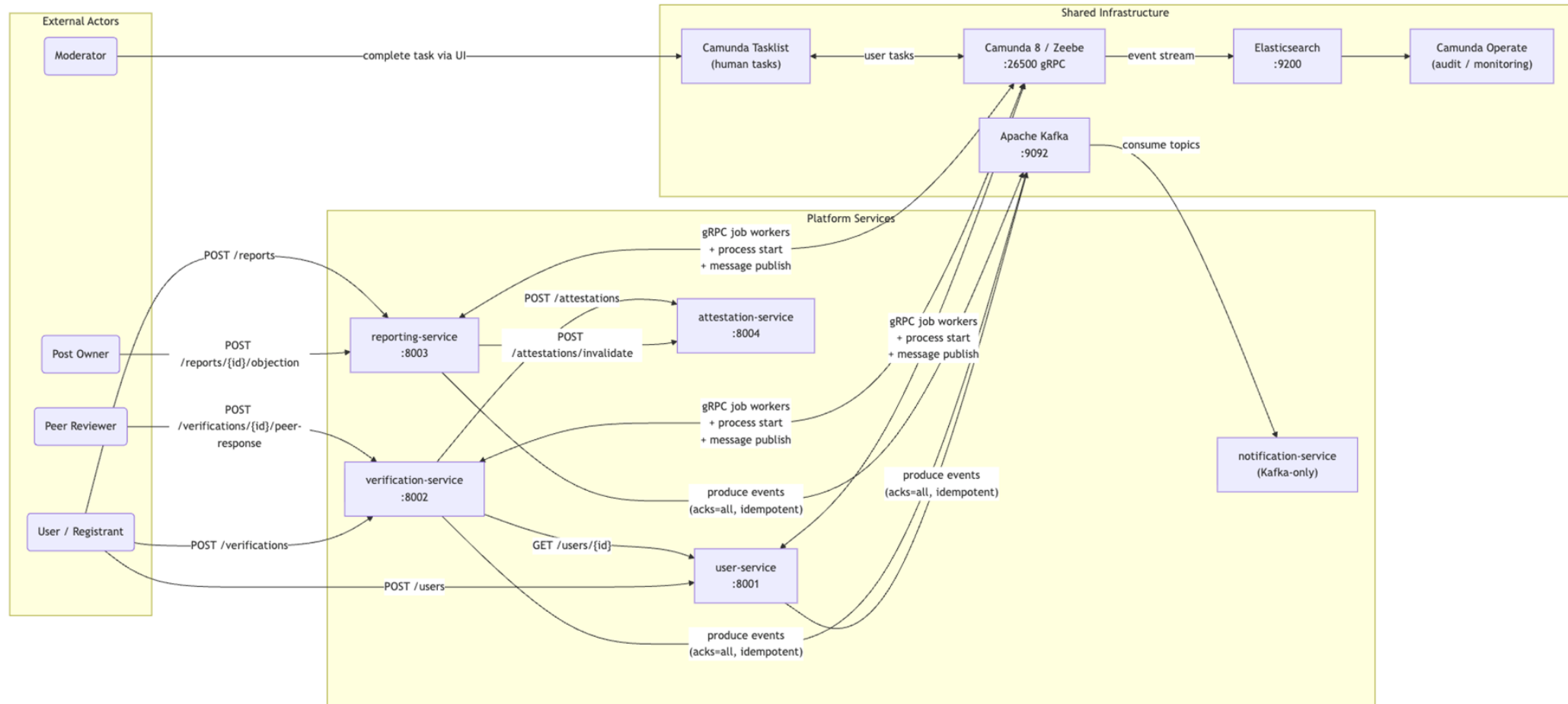


Appendix — Architecture Bounded Context



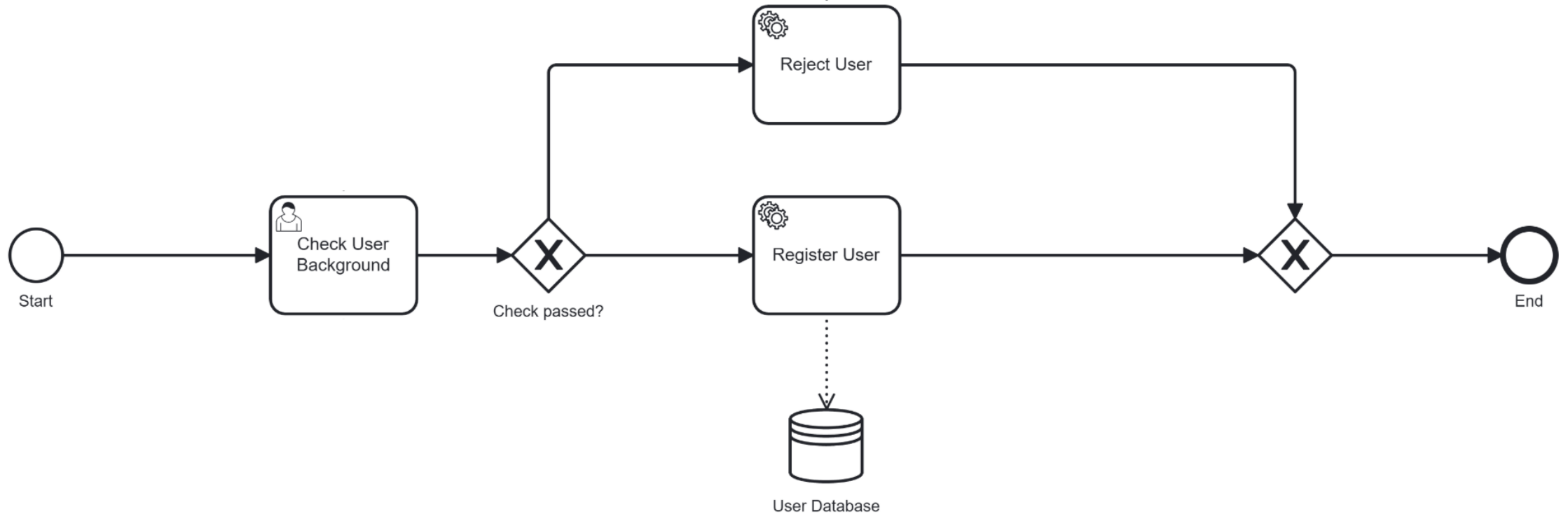


Appendix — Architecture Service Diagram



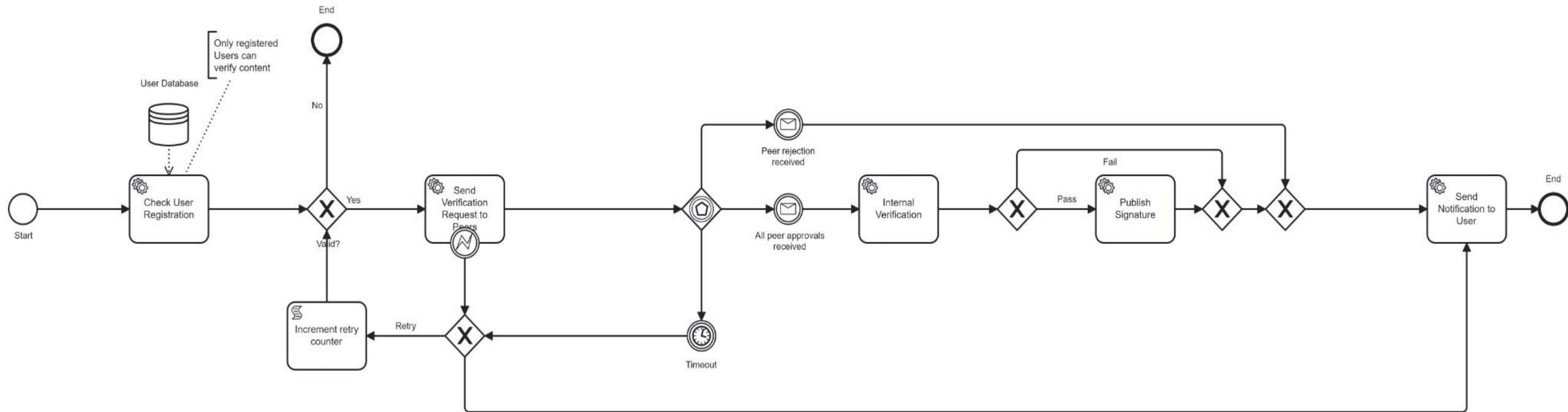


Appendix — Register User Process





Appendix — Verify Content Process





Appendix — Report Content Process

